

CampaignIQ

Pandas · Génération de données · PostgreSQL · Agrégation marketing

De la donnée brute à l'analyse complète :
générer un dataset réaliste avec Faker, le nettoyer avec pandas,
calculer les KPIs marketing et l'exporter dans PostgreSQL.

ESD Lyon

M1 Data - Workshop Programmation - Analyse de données

Pierre LACOGNATA (pierre@lacognata.io)

Ressources : [par ici](#)

Année scolaire : 2025 – 2026



lacognata.io

Faker

numpy

pandas

dropna · fillna

groupby · pivot_table

resample

SQLAlchemy

PostgreSQL





Table des matières

01	Contexte du projet — NovaBrand et ses campagnes	3
	1.1 Présentation de NovaBrand ·	
	1.2 Ce que l'on va construire ·	
	1.3 Structure du dataset	
02	 Setup et génération du dataset avec Faker	4–6
	2.1 Installer les bibliothèques ·	
	2.2 Comprendre Faker ·	
	2.3 Générer les campagnes ·	
	2.4 Sauvegarder en CSV	
03	 Chargement et exploration du dataset	7–8
	3.1 Lire le CSV avec pandas ·	
	3.2 Explorer les données ·	
	3.3 Détecter les problèmes	
04	 Nettoyage et transformation	9–12
	4.1 Valeurs manquantes ·	
	4.2 Doublons ·	
	4.3 Types de données ·	
	4.4 Valeurs aberrantes ·	
	4.5 Colonnes calculées	
05	 Agrégation et analyse	13–16
	5.1 groupby ·	
	5.2 pivot_table ·	
	5.3 resample — analyse temporelle ·	
	5.4 Résumé KPIs	
06	 Export vers PostgreSQL	17–19
	6.1 PostgreSQL Portable — rappel installation ·	
	6.2 SQLAlchemy — connexion ·	
	6.3 to_sql — charger le DataFrame ·	
	6.4 Vérification SQL	
07	Checklist et récapitulatif	20

Contexte du projet – NovaBrand

1.1 Présentation de NovaBrand

NovaBrand est une entreprise fictive spécialisée dans la vente de produits grand public en ligne. Elle opère dans plusieurs pays européens et investit chaque mois des budgets significatifs dans des campagnes marketing multi-canaux. Son équipe data a besoin d'analyser la performance de ces campagnes pour orienter les décisions budgétaires du trimestre prochain.

C'est vous qui jouez le rôle de l'équipe data. Vous allez d'abord **générer un dataset réaliste** qui simule les données brutes exportées depuis leur outil de gestion de campagnes, puis vous allez le **nettoyer, le transformer et l'analyser avec pandas**, avant de le charger dans **PostgreSQL** pour permettre des requêtes SQL ultérieures.

1.2 Ce que l'on va construire

PIPELINE DU PROJET CAMPAIGNIQ

Demi-journée 1

- ▶ Générer le dataset avec Faker + numpy
- ▶ Introduire des problèmes de qualité intentionnels
- ▶ Sauvegarder en `campaigns.csv`

Demi-journée 2

- ▶ Valeurs manquantes — dropna / fillna
- ▶ Doublons — drop_duplicates
- ▶ Types incorrects — to_datetime
- ▶ KPIs — CTR / CAC / ROAS / CVR

Demi-journée 3

- ▶ groupby multi-niveaux
- ▶ pivot_table canal × segment
- ▶ resample mensuel et hebdo
- ▶ Export PostgreSQL via SQLAlchemy

Setup et génération du dataset

2.1 Installer les bibliothèques

Ouvrez VSCode dans un nouveau dossier `campaigniq/`. Créez et activez votre environnement virtuel, puis installez les bibliothèques nécessaires.

```
$ 128187 Terminal PowerShell dans le dossier campaigniq

cd C:\Users\VotreNom\Documents\campaigniq
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install pandas faker numpy sqlalchemy psycopy2-binary openpyxl
```

🔗 PSYCOPG2-BINARY

`psycopy2-binary` est le pilote Python pour PostgreSQL. Il permet à SQLAlchemy de communiquer avec votre base de données PostgreSQL Portable. Il faut l'installer même si vous ne vous connectez pas à PostgreSQL immédiatement — il sera utilisé au chapitre 6.

2.2 Comprendre Faker

Faker est une bibliothèque Python qui génère des données fictives réalistes : noms, adresses, emails, dates, textes... Elle est utilisée massivement dans les entreprises pour créer des jeux de données de test. Voici comment elle fonctionne :

```
from faker import Faker

fake = Faker('fr_FR')          # locale française

# Générer différents types de données
print(fake.name())             # 'Marie Dupont'
print(fake.company())          # 'Groupe Martin SAS'
print(fake.date_between(      # date aléatoire entre 2 dates
    start_date='-1y',
    end_date='today'
))                             # '2024-07-23'
print(fake.random_element(     # choisir aléatoirement dans une liste
    elements=['email', 'social', 'search']
))
```

Pour notre projet, on n'utilise pas la locale française car les données marketing sont en anglais (noms de campagnes, canaux, segments). On utilise la locale par défaut `'en_US'`.

2.3 Générer le dataset — script complet

Créez un fichier `generate_data.py` dans votre dossier `campaigniq/`. Ce script génère 5 000 lignes de données de campagnes marketing, avec des **problèmes de qualité intentionnels** pour l'exercice de nettoyage.

```
generate_data.py

import pandas as pd
import numpy as np
from faker import Faker
import random

# — Initialisation —
fake = Faker()
np.random.seed(42)    # reproductibilité : même seed = mêmes données
random.seed(42)
N = 5000              # nombre de campagnes à générer

# — Listes de valeurs possibles —
channels = ['email', 'social', 'search', 'display']
countries = ['France', 'Germany', 'Spain', 'Italy', 'Netherlands']
segments = ['new', 'returning', 'vip']

# — Génération des données de base —
rows = []
for i in range(N):
    channel = random.choice(channels)
    country = random.choice(countries)
    segment = random.choice(segments)

    # Impressions selon le canal (search/display ont plus d'impressions)
    if channel in ['search', 'display']:
        impressions = int(np.random.lognormal(10.5, 0.8))
    else:
        impressions = int(np.random.lognormal(9.5, 0.7))

    # CTR (taux de clic) réaliste par canal
    ctr_base = {'email': 0.18, 'social': 0.04,
                'search': 0.08, 'display': 0.002}
    ctr = max(0, np.random.normal(ctr_base[channel], ctr_base[channel]*0.3))
    clicks = int(impressions * ctr)

    # Taux de conversion et spend
    conv_rate = max(0, np.random.normal(0.03, 0.01))
    conversions = int(clicks * conv_rate)
    spend = round(np.random.lognormal(7, 0.6), 2)
    revenue = round(conversions * np.random.uniform(80, 150), 2)

    rows.append({
        'campaign_id' : f'CAMP-{i+1:05d}',
        'date'        : fake.date_between(start_date='-2y', end_date='today'),
        'channel'      : channel,
        'campaign_name': f'{fake.catch_phrase()} {channel.title()} {country[:2].upper()}',
        'country'      : country,
        'segment'      : segment,
        'impressions'  : impressions,
        'clicks'       : clicks,
        'conversions'  : conversions,
        'spend'        : spend,
        'revenue'      : revenue,
    })
```

```
df = pd.DataFrame(rows)
```

2.4 Introduire des problèmes de qualité intentionnels

Les vraies données marketing sont rarement propres. Elles contiennent des valeurs manquantes (l'outil de tracking a planté), des doublons (exports en double), des types incorrects. On va les simuler **après** avoir généré les données propres.

Continuez dans le même fichier `generate_data.py`, sous la boucle :

```
generate_data.py - suite (après la boucle)

# — Introduire des problèmes de qualité —

# 1. Valeurs manquantes dans 'spend' et 'country' (~5% des lignes)
idx_spend = np.random.choice(df.index, size=int(N*0.05), replace=False)
idx_country = np.random.choice(df.index, size=int(N*0.04), replace=False)
df.loc[idx_spend, 'spend'] = np.nan
df.loc[idx_country, 'country'] = np.nan

# 2. Doublons - dupliquer 50 lignes aléatoires
duplicates = df.sample(50, random_state=42)
df = pd.concat([df, duplicates], ignore_index=True)

# 3. Valeurs aberrantes dans 'clicks' - quelques valeurs impossibles
idx_aberrant = np.random.choice(df.index, size=10, replace=False)
df.loc[idx_aberrant, 'clicks'] = -999 # clics négatifs → impossible

# 4. Type incorrect - date stockée comme chaîne dans quelques lignes
df['date'] = df['date'].astype(str) # tout en string pour simuler un export CSV

# — Mélanger et sauvegarder —
df = df.sample(frac=1, random_state=42).reset_index(drop=True)
df.to_csv('campaigns.csv', index=False, encoding='utf-8')

print(f"Dataset généré : {len(df)} lignes × {len(df.columns)} colonnes")
print(f"Fichier sauvegardé : campaigns.csv")
print(df.head(3))
```

```
$ 128187 Lancer la génération

python generate_data.py
# Sortie attendue :
# Dataset généré : 5050 lignes × 11 colonnes
# Fichier sauvegardé : campaigns.csv
```

💡 POURQUOI NP.RANDOM.SEED(42) ?

La graine (seed) garantit que les données aléatoires sont **toujours identiques** d'une exécution à l'autre. Tout le monde dans la salle obtient exactement le même fichier `campaigns.csv`. Sans seed, chaque exécution donnerait un fichier différent et les résultats des analyses varieraient d'un poste à l'autre.

⚠️ 5 050 LIGNES ET PAS 5 000 ?

Normal — on a ajouté 50 lignes dupliquées intentionnellement. Le nettoyage ramènera le dataset à 5 000 lignes propres au chapitre 4. C'est exactement le comportement que l'on veut observer.

Chargement et exploration du dataset

3.1 Lire le CSV avec pandas

Créez un nouveau fichier `analyse.py` dans le même dossier. Tous les chapitres suivants s'écrivent dans ce fichier.

```
analyse.py — début du fichier

import pandas as pd
import numpy as np

# Lire le CSV généré
df = pd.read_csv('campaigns.csv')

print("Dimensions :", df.shape)      # (5050, 11)
print(df.head())                     # 5 premières lignes
```

3.2 Explorer les données — les 4 réflexes essentiels

Avant de nettoyer quoi que ce soit, on explore. Ces quatre commandes sont à exécuter **systématiquement** à chaque fois qu'on reçoit un nouveau dataset.

```
# — Réflexe 1 : vue d'ensemble —————
df.info()
# Affiche : nom de chaque colonne, type détecté, nombre de valeurs non-nulles
# C'est ici qu'on voit si une colonne numérique a été lue en object (str)

# — Réflexe 2 : statistiques descriptives —————
df.describe()
# count / mean / std / min / 25% / 50% / 75% / max
# On cherche des min négatifs (clics négatifs), des max aberrants, etc.

# — Réflexe 3 : valeurs manquantes —————
df.isnull().sum()
# Nombre de NaN par colonne — ici on doit voir ~250 NaN dans spend,
# ~200 NaN dans country

# — Réflexe 4 : doublons —————
df.duplicated().sum()
# Nombre de lignes identiques — doit afficher 50
```

3.3 Interpréter les résultats

Après avoir lancé ces quatre commandes, vous devriez observer :

<code>date</code> est de type <code>object</code>	La date est stockée comme texte	<code>pd.to_datetime()</code>
<code>spend</code> a ~250 NaN	Valeurs manquantes dans le budget	<code>fillna()</code> avec la médiane
<code>country</code> a ~200 NaN	Pays inconnu pour certaines lignes	<code>dropna()</code> ou <code>fillna('Unknown')</code>
<code>clicks</code> min = -999	Valeurs aberrantes (clics négatifs)	Filtrer avec condition booléenne
<code>uplicated() = 50</code>	50 lignes en double	<code>drop_duplicates()</code>

📌 **DF.INFO() VS DF.DESCRIBE()**

`df.info()` donne la *structure* : types, mémoire, valeurs non-nulles. `df.describe()` donne la *distribution statistique* des colonnes numériques. Les deux sont complémentaires et indispensables. Dans une vraie mission data, vous les lanceriez tous les deux dès réception d'un fichier.

Nettoyage et transformation

4.1 Corriger les types — la date en premier

La colonne `date` a été lue comme texte (`object`). pandas ne peut pas faire d'analyse temporelle (resample, extraction de mois, de semaine) si la date est une chaîne. On la convertit en datetime.

```
# Convertir la colonne date en datetime
df['date'] = pd.to_datetime(df['date'])

# Vérification
print(df['date'].dtype)      # datetime64[ns] ✓
print(df['date'].min())      # date la plus ancienne
print(df['date'].max())      # date la plus récente
```

📌 PD.TO_DATETIME() — FORMATS ACCEPTÉS

pandas reconnaît automatiquement les formats courants : `'2024-03-15'`, `'15/03/2024'`, `'March 15, 2024'`. Si votre format est ambigu (ex : `'01/02/2024'` — janvier ou février ?), précisez avec `format='%d/%m/%Y'`.

4.2 Valeurs manquantes — dropna et fillna

Il existe deux stratégies pour gérer les valeurs manquantes : les **supprimer** (`dropna`) ou les **remplacer** (`fillna`). Le choix dépend du contexte métier.

```
# — Stratégie 1 : supprimer les lignes où country est inconnu —
# On ne peut pas analyser les performances par pays sans le pays
df = df.dropna(subset=['country'])
print("Après dropna country :", len(df), "lignes")

# — Stratégie 2 : remplacer spend manquant par la médiane —
# On ne veut pas perdre des lignes pour un budget manquant.
# La médiane est plus robuste que la moyenne face aux valeurs extrêmes.
mediane_spend = df['spend'].median()
df['spend'] = df['spend'].fillna(mediane_spend)
print(f"spend NaN remplacés par la médiane : {mediane_spend:.2f} €")

# Vérification — plus aucun NaN
print(df[['spend', 'country']].isnull().sum())
```

💡 MÉDIANE VS MOYENNE POUR FILLNA

Si `spend` a quelques valeurs très élevées (campagnes exceptionnelles à 50 000 €), la **moyenne** sera tirée vers le haut et ne sera pas représentative d'une campagne "normale". La **médiane** — valeur centrale — est insensible aux extrêmes. En data marketing, préférez la médiane pour les montants.

4.3 Supprimer les doublons

```
# Supprimer les 50 lignes dupliquées
df = df.drop_duplicates()
df = df.reset_index(drop=True) # réindexer proprement après suppression
print("Après dédoublonnage :", len(df), "lignes")

# Vérification
print("Doublons restants :", df.duplicated().sum()) # doit afficher 0
```

4.4 Valeurs aberrantes — filtrage conditionnel

Les clics négatifs sont physiquement impossibles. On les filtre avec une condition booléenne. On en profite pour vérifier aussi que les impressions sont cohérentes.

```
# Compter les lignes aberrantes avant filtrage
print("Clics négatifs :", (df['clicks'] < 0).sum()) # 10 lignes
print("Clics > impressions :", (df['clicks'] > df['impressions']).sum())

# Filtrer : garder seulement les lignes valides
df = df[df['clicks'] ≥ 0]
df = df[df['clicks'] ≤ df['impressions']]
df = df.reset_index(drop=True)
print("Après nettoyage aberrations :", len(df), "lignes")
```

4.5 Colonnes calculées — les KPIs marketing

Une fois les données propres, on calcule les indicateurs clés de performance. Ces colonnes n'existent pas dans le dataset brut — elles se déduisent des colonnes existantes.

KPI	Formule	Interprétation
CTR — Click-Through Rate	clicks / impressions	% de personnes qui ont cliqué. Email ~18%, Display ~0.2%
CAC — Coût d'Acquisition	spend / conversions	Combien coûte chaque client acquis. Plus c'est bas, mieux c'est.
ROAS — Return On Ad Spend	revenue / spend	Revenus générés par euro investi. ROAS > 1 = rentable.
CVR — Conversion Rate	conversions / clicks	% de cliqueurs qui achètent. Indicateur de qualité du trafic.

```
# — Calcul des KPIs —————

# CTR — éviter la division par zéro avec np.where
df['ctr'] = np.where(
    df['impressions'] > 0,
    df['clicks'] / df['impressions'],
    0
)

# CAC — coût par conversion (NaN si 0 conversions — normal, on garde)
df['cac'] = df['spend'] / df['conversions'].replace(0, np.nan)

# ROAS — retour sur investissement publicitaire
df['roas'] = df['revenue'] / df['spend'].replace(0, np.nan)

# CVR — taux de conversion
df['cvr'] = np.where(
    df['clicks'] > 0,
    df['conversions'] / df['clicks'],
    0
)

# Vérification des nouvelles colonnes
print(df[['ctr', 'cac', 'roas', 'cvr']].describe().round(3))
```

📌 NP.WHERE — ÉVITER LES DIVISIONS PAR ZÉRO

`np.where(condition, valeur_si_vrai, valeur_si_faux)`. Indispensable pour les KPIs marketing où le dénominateur peut être zéro. Sans cette précaution, une campagne avec 0 impressions génèrerait une erreur ou un `inf` qui fausserait toutes les statistiques.

4.6 Extraire des colonnes temporelles

Depuis une colonne datetime, pandas permet d'extraire directement l'année, le mois, la semaine, le jour de la semaine. Ces colonnes seront utiles pour les agrégations temporelles du chapitre 5.

```
# Extraire des composantes temporelles depuis la date
df['year']      = df['date'].dt.year
df['month']     = df['date'].dt.month
df['week']      = df['date'].dt.isocalendar().week.astype(int)
df['day_of_week'] = df['date'].dt.day_name() # 'Monday', 'Tuesday', etc.

# Sauvegarder le dataset nettoyé et enrichi
df.to_csv('campaigns_clean.csv', index=False)
print(f"Dataset nettoyé : {len(df)} lignes, {len(df.columns)} colonnes")
print("Colonnes :", df.columns.tolist())
```

💡 BILAN DU NETTOYAGE — CE QU'ON A FAIT

- Types corrigés : date en datetime ✓
- Valeurs manquantes : country supprimé (~200 lignes), spend remplacé par médiane ✓
- Doublons supprimés : 50 lignes retirées ✓
- Aberrations filtrées : clics négatifs et incohérents retirés ✓
- KPIs calculés : CTR, CAC, ROAS, CVR ✓
- Colonnes temporelles ajoutées : year, month, week, day_of_week ✓

Agrégation et analyse

5.1 groupby — agrégations par groupe

`groupby` est l'opération fondamentale de l'analyse de données : on regroupe les lignes par une ou plusieurs colonnes, puis on applique une fonction d'agrégation (somme, moyenne, compte, etc.). C'est l'équivalent pandas d'un `GROUP BY` en SQL.

```
# — Agrégation simple : performance par canal —————
perf_canal = df.groupby('channel').agg(
    nb_campagnes    = ('campaign_id', 'count'),
    spend_total     = ('spend',       'sum'),
    revenue_total   = ('revenue',    'sum'),
    conversions     = ('conversions', 'sum'),
    ctr_moyen       = ('ctr',         'mean'),
    roas_moyen      = ('roas',        'mean'),
).round(2)

print(perf_canal.sort_values('roas_moyen', ascending=False))
```

```
# — Agrégation multi-niveaux : canal × pays —————
# On regroupe sur deux colonnes simultanément
perf_canal_pays = df.groupby(['channel', 'country']).agg(
    spend_total     = ('spend',       'sum'),
    revenue_total   = ('revenue',    'sum'),
    roas_moyen      = ('roas',        'mean'),
).round(2)

# Trouver le meilleur ROAS par pays (indépendamment du canal)
meilleur_roas_pays = df.groupby('country')['roas'].mean().sort_values(ascending=False)
print(meilleur_roas_pays)
```

```
# — Top 5 campagnes par revenue —————
cols = ['campaign_name', 'channel', 'country', 'revenue', 'roas']
top5 = df.nlargest(5, 'revenue')[cols]
print(top5)
```

5.2 pivot_table — tableaux croisés

`pivot_table` permet de créer un tableau croisé dynamique — l'équivalent pandas d'un tableau croisé Excel. On met une variable en ligne, une en colonne, et une valeur agrégée à l'intersection.

```
# Tableau croisé : canal (lignes) × segment (colonnes) → ROAS moyen
tableau_roas = pd.pivot_table(
    df,
    values='roas',
    index='channel',
    columns='segment',
```

```

        aggfunc='mean'
    ).round(2)

    print(tableau_roas)
#           new  returning  vip
# display  3.12      3.45  4.01
# email    4.23      4.87  5.34
# search   3.78      4.12  4.56
# social   2.90      3.21  3.88

# Spend total par canal x pays
tableau_spend = pd.pivot_table(
    df,
    values='spend',
    index='channel',
    columns='country',
    aggfunc='sum',
    margins=True,          # ajouter une ligne et colonne "Total"
    margins_name='Total'
).round(0)

print(tableau_spend)

```


5.3 resample — analyse temporelle

`resample` fonctionne comme un `groupby` mais sur des intervalles de temps. Il faut d'abord que la colonne `date` soit définie comme **index** du DataFrame. Ensuite, on peut agréger par semaine, par mois, par trimestre, etc.

```
# Définir la date comme index pour resample
df_ts = df.set_index('date').sort_index()

# — Revenus par semaine —————
revenus_semaine = df_ts['revenue'].resample('W').sum()
print(revenus_semaine.tail(8))      # 8 dernières semaines

# — Spend et revenue par mois —————
mensuel = df_ts[['spend', 'revenue', 'conversions']].resample('ME').sum()
# 'ME' = Month End — dernier jour du mois (pandas ≥ 2.2)
# Ancienne syntaxe : 'M' (toujours valide)

# Calculer le ROAS mensuel depuis les totaux mensuels
mensuel['roas_mensuel'] = (mensuel['revenue'] / mensuel['spend']).round(2)
print(mensuel.tail(6))              # 6 derniers mois

# — Croiser resample et groupby —————
# Revenus mensuels par canal — un peu plus avancé
mensuel_canal = (
    df.groupby([pd.Grouper(key='date', freq='ME'), 'channel'])
    ['revenue'].sum().unstack()
)
print(mensuel_canal.tail(3))

# channel    display    email    search    social
# date
# 2024-10-31  12450.0   45230.0  28900.0  18700.0
# 2024-11-30  11200.0   48100.0  31200.0  17400.0
# 2024-12-31  13800.0   52400.0  29700.0  20100.0
```

📌 CODES DE FRÉQUENCE RESAMPLE

'D' = jour · 'W' = semaine · 'ME' = fin de mois · 'QE' = fin de trimestre · 'YE' = fin d'année.
Pour la compatibilité avec les versions plus anciennes de pandas, 'M', 'Q', 'Y' fonctionnent aussi.

5.4 Résumé KPIs — rapport final

```
# — Rapport de synthèse complet —————
print("="*50)
print("RAPPORT CAMPAIGNIQ — SYNTHÈSE")
print("="*50)
print(f"Période analysée : {df['date'].min().date()} → {df['date'].max().date()}")
print(f"Nombre de campagnes      : {len(df):,}")
print(f"Spend total                  : {df['spend'].sum():.0f} €")
print(f"Revenue total                : {df['revenue'].sum():.0f} €")
print(f"ROAS global                  : {df['revenue'].sum()/df['spend'].sum():.2f}")
print(f"Total conversions            : {df['conversions'].sum():,}")
```

```
print(f"CAC moyen          : {df['cac'].median():.2f} €")
print(f"Meilleur canal (ROAS) : {df.groupby('channel')['roas'].mean().idxmax()}")
print(f"Meilleur pays (revenue): {df.groupby('country')['revenue'].sum().idxmax()}")
```

Export vers PostgreSQL

6.1 PostgreSQL Portable — rappel installation et lancement

Si PostgreSQL Portable n'est pas encore installé sur votre poste, voici la procédure complète. Si c'est déjà fait, passez directement au point 6.2.

- 1 **Télécharger l'archive** fournie par l'enseignant (PostgreSQL Portable avec binaires récents). Décompresser dans `C:\postgresql-portable\`. Éviter les chemins avec espaces.
- 2 **Lancer PostgreSQL** : double-cliquer sur `PostgreSQLPortable.exe` dans le dossier décompressé. Une fenêtre noire s'ouvre — c'est le terminal psql. Le serveur PostgreSQL tourne maintenant en arrière-plan.
- 3 **Vérifier la connexion** : dans la fenêtre psql, taper `\l` puis Entrée. Vous devez voir la liste des bases de données (au minimum `postgres`).
- 4 **Créer la base de données du workshop** dans la fenêtre psql :

```
$ 128187 Dans la fenêtre psql (PostgreSQL Portable)

CREATE DATABASE campaigniq;
\c campaigniq
-- Vous êtes maintenant connecté à la base campaigniq
-- Sortie attendue : "You are now connected to database "campaigniq"
```

PARAMÈTRES DE CONNEXION PAR DÉFAUT (POSTGRESQL PORTABLE)

Hôte : localhost · **Port** : 5432 · **Utilisateur** : postgres · **Mot de passe** : (vide) · **Base** : campaigniq
Ces paramètres sont ceux que vous utiliserez dans SQLAlchemy au point suivant.

6.2 SQLAlchemy — créer le moteur de connexion

SQLAlchemy est la bibliothèque standard Python pour communiquer avec des bases de données relationnelles. Elle permet d'utiliser le même code Python quelle que soit la base de données (PostgreSQL, SQLite, MySQL...). On crée un **moteur** (*engine*) qui représente la connexion.

```
from sqlalchemy import create_engine, text

# — Créer le moteur de connexion à PostgreSQL Portable —
# Format de l'URL : dialect+driver://user:password@host:port/database
engine = create_engine(
    "postgresql+psycopg2://postgres:@localhost:5432/campaigniq"
)

#
#
```

	dialect	user	pwd	hôte	port	base

```
# Tester la connexion
try:
    with engine.connect() as conn:
        result = conn.execute(text("SELECT version();"))
        print("Connexion OK -", result.fetchone()[0])
except Exception as e:
    print("Erreur de connexion :", e)
```

⚠ ERREUR DE CONNEXION — CAUSES FRÉQUENTES

- **PostgreSQLPortable.exe non lancé** → lancer le .exe en premier
- **Base campaigniq inexistante** → créer la base dans psql avec `CREATE DATABASE campaigniq;`
- **Port 5432 déjà occupé** → une autre instance de PostgreSQL tourne peut-être. Fermer les autres instances.

6.3 to_sql — charger le DataFrame dans PostgreSQL

La méthode `to_sql()` de pandas permet d'insérer directement un DataFrame dans une table PostgreSQL. C'est l'opération inverse de `read_sql()`.

```
# — Charger le DataFrame nettoyé dans PostgreSQL —————
df.to_sql(
    name='campaigns',      # nom de la table à créer
    con=engine,            # moteur SQLAlchemy
    if_exists='replace',   # 'replace' : supprimer et recréer si existe déjà
    index=False,          # ne pas écrire l'index pandas comme colonne
    chunksize=500,        # insérer par lots de 500 lignes (plus stable)
    method='multi'         # insertion multi-valeurs — plus rapide
)
print("Table 'campaigns' chargée dans PostgreSQL ✅")
```

📌 IF_EXISTS — TROIS OPTIONS

- `'fail'` : erreur si la table existe déjà (comportement par défaut)
- `'replace'` : supprimer la table et la recréer — ce qu'on utilise ici
- `'append'` : ajouter les lignes sans toucher à ce qui existe

En workshop, on utilise `'replace'` pour pouvoir relancer le script plusieurs fois sans erreur.

6.4 Vérification dans psql et depuis Python

On vérifie que les données ont bien été chargées — d'abord depuis la fenêtre psql, puis depuis Python.

```
$ 128187 Dans la fenêtre psql vérification

\dt
-- Doit afficher la table "campaigns"

SELECT COUNT(*) FROM campaigns;
-- Doit afficher ~4 700 lignes (selon le nettoyage effectué)

SELECT channel, COUNT(*), ROUND(AVG(roas)::numeric, 2) AS roas_moyen
FROM campaigns
GROUP BY channel
ORDER BY roas_moyen DESC;
-- Résultats de l'analyse par canal, directement en SQL
```

```
# — Vérification depuis Python — relire depuis PostgreSQL —————
sql = (
    "SELECT channel, COUNT(*) as nb,"
    "ROUND(AVG(roas)::numeric, 2) as roas_moyen "
    "FROM campaigns GROUP BY channel ORDER BY roas_moyen DESC"
)
df_check = pd.read_sql(sql,
    con=engine
```

```
)  
print(df_check)  
  
# Compter le nombre total de lignes  
nb_lignes = pd.read_sql("SELECT COUNT(*) as total FROM campaigns", con=engine)  
print(f"Total lignes en base : {nb_lignes['total'][0]}")
```

💡 POURQUOI CHARGER DANS POSTGRESQL SI ON A DÉJÀ PANDAS ?

Deux raisons importantes. Premièrement, en entreprise, plusieurs personnes ont besoin d'accéder aux données analysées — un tableau de bord Power BI, un autre analyste, un développeur. PostgreSQL permet ce partage. Deuxièmement, pour des datasets de plusieurs millions de lignes, SQL est souvent plus rapide que pandas pour les agrégations. On verra cela concrètement au Projet 3 avec le dataset Online Retail II.

6.5 Fermer le moteur proprement

```
# Toujours fermer le moteur en fin de script  
engine.dispose()  
print("Connexion fermée.")
```

Checklist et récapitulatif

Bilan des compétences — CampaignIQ

À la fin de ce projet, vous devez être capables d'effectuer chacune de ces opérations de façon autonome. Cochez au fur et à mesure.

Compétence	Commande clé	✓
Générer un dataset réaliste avec Faker et numpy	<code>Faker()</code> <code>np.random.lognormal()</code>	
Lire un CSV avec pandas	<code>pd.read_csv('fichier.csv')</code>	
Explorer un dataset inconnu (4 réflexes)	<code>df.info()</code> <code>df.describe()</code> <code>df.isnull().sum()</code> <code>df.duplicated().sum()</code>	
Corriger le type d'une colonne date	<code>pd.to_datetime(df['date'])</code>	
Supprimer les lignes avec valeurs manquantes	<code>df.dropna(subset=['col'])</code>	
Remplacer les valeurs manquantes par la médiane	<code>df['col'].fillna(df['col'].median())</code>	
Supprimer les doublons	<code>df.drop_duplicates()</code>	
Filtrer des valeurs aberrantes	<code>df[df['col'] ≥ 0]</code>	
Calculer des colonnes dérivées (KPIs)	<code>df['ctr'] = df['clicks'] / df['impressions']</code>	
Extraire des composantes temporelles	<code>df['date'].dt.year</code> <code>.dt.month</code> <code>.dt.isocalendar().week</code>	
Agréger par groupe avec groupby	<code>df.groupby('canal').agg(spend=('spend', 'sum'))</code>	
Créer un tableau croisé	<code>pd.pivot_table(df, values=..., index=..., columns=...)</code>	

Agréger par période temporelle	<code>df.set_index('date').resample('ME').sum()</code>
Connexion SQLAlchemy à PostgreSQL	<code>create_engine("postgresql+psycopg2://postgres:@localhost:5432/db")</code>
Charger un DataFrame dans PostgreSQL	<code>df.to_sql('table', engine, if_exists='replace')</code>
Lire depuis PostgreSQL dans pandas	<code>pd.read_sql("SELECT ...", engine)</code>

Récapitulatif — pipeline complet

CE QUE VOUS AVEZ CONSTRUIT

generate_data.py

- Faker + numpy
- 5 000 lignes générées
- Problèmes qualité intentionnels

`campaigns.csv`

analyse.py

- `pd.read_csv()` · `info()` · `describe()`
- Nettoyage complet
- KPIs calculés
- `groupby` · `pivot_table` · `resample`
- `to_sql()` → PostgreSQL

PostgreSQL

- Base : campaigniq
- Table : campaigns
- ~4 700 lignes propres
- Requêtes SQL analytiques